

Ideias para circuit breaker

Sugestões de circuit breaker

O circuit breaker controla cada provedor separadamente. Quando um deles falha repetidamente, o sistema o suspende por um período e segue direto para o próximo. Após esse período, permite uma chamada de teste para verificar se o provedor se recuperou.

Opção	Como funcionaria	Vantagens	Desvantagens	Adequação ao projeto
Sem circuit breaker: timeout, retry e fallback	Mantém as tentativas atuais. Se o Provedor A falhar, tenta o B.	Simples e não exige infraestrutura extra.	Toda requisição ainda pode esperar pelo timeout e pelas tentativas do provedor indisponível.	Boa opção inicial para o home test e para baixo volume.
Circuit breaker em memória	Guarda o estado do provedor na memória do processo ou em cache local, como APCu.	Fácil de implementar e não requer Redis.	Cada processo ou container mantém seu próprio estado; reiniciar pode apagar os dados.	Serve para desenvolvimento ou uma única instância. Não é consistente com múltiplos containers.
Circuit breaker compartilhado com Redis	Armazena contadores e estados no Redis. Todas as instâncias consultam o mesmo estado para cada provedor. O Laravel oferece suporte a cache Redis e locks atômicos. Documentação do Laravel	Compartilha o estado entre containers e evita que cada instância continue chamando um provedor indisponível.	Adiciona Redis à operação. É preciso tratar concorrência e decidir o comportamento se o Redis ficar indisponível.	Opção recomendada para produção com múltiplas instâncias , caso Redis já faça parte da operação.
Biblioteca PHP com	Usa uma biblioteca,	Evita implementar	Acrescenta uma dependência e	Alternativa ao circuito próprio

Opção	Como funcionaria	Vantagens	Desvantagens	Adequação ao projeto
armazenamento Redis	como <u>Ganesha</u> , para gerenciar os estados e estratégias, com Redis como armazenamento compartilhado.	toda a máquina de estados manualmente; oferece estratégias e adaptadores de armazenamento.	exige validar compatibilidade, manutenção e integração com o fluxo Laravel atual.	com Redis. Vale avaliar antes de adotar.
Circuit breaker no gateway ou service mesh	Um componente externo, como gateway ou service mesh, controla as chamadas e suspende os destinos com falha.	Pode centralizar resiliência para vários serviços e aplicações.	Traz configuração e operação adicionais para uma necessidade que o projeto ainda não tem.	Mais adequado quando a arquitetura já usa esse tipo de componente; excessivo para o escopo atual.

Recomendação

Para o projeto atual, eu manteria timeout, retry e fallback como estão. Se o serviço passar a rodar com várias instâncias e houver necessidade de reduzir chamadas repetidas a provedores fora do ar, adicionaria um circuit breaker compartilhado por Redis.

Um circuito em memória pode ser um passo simples para desenvolvimento, mas cada container teria um estado independente. Para compartilhar contadores e expiração com segurança, convém usar operações atômicas; o Redis documenta padrões com **INCR** e alerta para a necessidade de combinar contador e expiração de forma segura. [Documentação do Redis sobre INCR](#)

Exemplo de política inicial

Os valores abaixo são um ponto de partida para medir e ajustar, não uma configuração universal.

Estado ou regra	Comportamento sugerido
Fechado (CLOSED)	Chamadas seguem normalmente para o provedor.
Abertura do circuito	Abrir após 5 falhas do provedor em uma janela de 30 segundos.

Estado ou regra	Comportamento sugerido
Aberto (OPEN)	Pular esse provedor imediatamente e tentar o próximo, sem esperar o timeout.
Tempo aberto	Manter o circuito aberto por 30 segundos antes de testar novamente.
Meio aberto (HALF_OPEN)	Permitir uma chamada de teste ao provedor após o tempo aberto.
Teste bem-sucedido	Fechar o circuito e zerar o contador de falhas.
Teste malsucedido	Reabrir o circuito e aguardar antes de um novo teste.

O circuito deve contar falhas de comunicação, timeout, respostas HTTP 5xx e respostas malformadas do provedor. Erros da requisição do cliente, como placa inválida, não devem afetar o circuito. Também é importante tratar separadamente erros de negócio, como tipo de débito desconhecido: eles não significam, por si só, que o provedor está indisponível.

Com os dois provedores indisponíveis ou com os circuitos abertos, a API pode retornar **503** rapidamente, sem repetir chamadas que provavelmente falharão.